



Web Fundamentals

WEF1VO

Teil 9: JavaScript im Browser

Medientechnik und -design | WS 2025/26

JavaScript-Objekte

Komplexe Datentypen with a Twist

Objekt

Eine Sammlung
zusammengehörender Daten.

Gehören zu den komplexen Datentypen.

Bilden i. d. R. Inhalte aus dem realen Leben ab.



Beispiel:

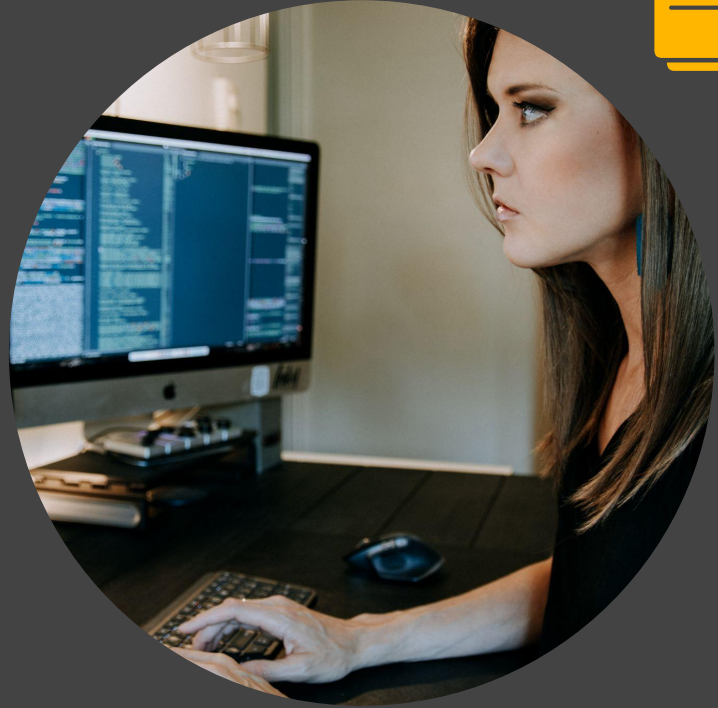
Student*in

Daten

- Name
- Alter
- Studiengang
- Beherrschte Programmiersprachen

Funktionalitäten

- Ausgabe der Programmiersprachen
- Erlernen von neuen Sprachen



Ansatz **ohne** Objekte



```
// Daten des*der Studenten*Studentin
let studentName = "John Doe";
let studentAge = 22;
let studentProgram = "MTD";
let studentLanguages = ["Java", "C#"];
```

```
// Ausgabe bzw. Änderung der Daten
function showLanguages() {
  console.log(`${studentName} (${studentAge}, ${studentProgram}) beherrscht
    diese Sprachen: ${studentLanguages}`);
}

function learnNewLanguage(language) {
  studentLanguages[studentLanguages.length] = language;
}
```

Aufrufen:

```
showLanguages(); // Java,C#
learnNewLanguage("JavaScript");
showLanguages(); // Java,C#,JavaScript
```

Objekt

Eine Datenkapsel
mit Eigenschaften
und Methoden



Ansatz als Objekt



```
const student = {  
  name: "John Doe",  
  age: 22,  
  program: "MTD",  
  languages: ["Java", "C#"],  
  showLanguages() {  
    console.log(`${this.name} (${this.age}, ${this.program}) beherrscht  
      diese Sprachen: ${this.languages}`);  
  },  
  learnNewLanguage(language) {  
    this.languages[this.languages.length] = language;  
  }  
};
```

Eigenschaften

Methoden



Objekte: Zugriff

`<Name des Objekts>.<Name der Eigenschaft>`

`<Name des Objekts>.<Name der Methode()>`

Aufrufen von Methoden

```
student.showLanguages(); // Java,C#  
student.learnNewLanguage("JavaScript");  
student.showLanguages(); // Java,C#,JavaScript
```

Ausgeben oder Verändern von Eigenschaften

```
console.log(student.name); // John Doe  
student.age = 23;
```

What's **this**?



```
showLanguages() {  
  console.log(`${this.name} (${this.age}, ${this.program}) beherrscht  
    diese Sprachen: ${this.languages}`);  
}
```

this: Enthält immer das aktuelle Objekt (hier **student**)



Eigene Objekte definieren

Objekt-Literal-
Schreibweise

Einzelne Objekte schnell
anlegen.

Konstruktorfunktionen

Erstellen von Objekten mit
einer speziellen
Erzeugerfunktion.

Klassensyntax

Objekte mit einer Syntax,
bekannt aus
klassenbasierten
Sprachen, erstellen.



Objekt-Literal-Syntax

```
const myObject = {  
  key1: value1,  
  key2: value2,  
  method1() { ... }  
};
```

Student (Objekt-Literal)



```
const student = {  
  name: "John Doe",  
  age: 22,  
  program: "MTD",  
  languages: ["Java", "C#"],  
  showLanguages() {  
    console.log(`${this.name} (${this.age}, ${this.program}) beherrscht  
      diese Sprachen: ${this.languages}`);  
  },  
  learnNewLanguage(language) {  
    this.languages[this.languages.length] = language;  
  }  
};
```



Konstruktor-Funktionen

```
function MyConstructorFunction(param1, param2) {  
    this.key1 = param1;  
    this.key2 = param2;  
    this.key3 = function() { ... }  
}
```

Student (Konstruktorfkt.)



```
function Student(name, age, program, languages) {  
  this.name = name;  
  this.age = age;  
  this.program = program;  
  this.languages = languages;  
  this.showLanguages = function () {  
    console.log(`${this.name} (${this.age}, ${this.program}) beherrscht  
      diese Sprachen: ${this.languages}`);  
  };  
  this.learnNewLanguage = function (language) {  
    this.languages[this.languages.length] = language;  
  };  
}
```

Konstruktorfkt.: Aufruf



```
const myObject1 = new MyConstructorFunction(arg1, arg2);  
const myObject2 = new MyConstructorFunction(arg3, arg4);
```

Studierenden-Objekte erzeugen

```
const student1 = new Student("John Doe", 22, "MTD", ["Java", "C#"]);  
const student2 = new Student("Jane Doe", 25, "SE", ["Pascal", "C", "Java"]);  
  
student1.showLanguages(); // Java,C#  
student1.learnNewLanguage("JavaScript");  
student1.showLanguages(); // Java,C#,JavaScript  
student2.showLanguages(); // Pascal,C,Java
```



Klassen -Syntax

```
class MyClass {  
    constructor(param1, param2) {  
        this.key1 = param1;  
        this.key2 = param2;  
    }  
    method1() { ... }  
}
```

Student (Klasse)



```
class Student {  
  constructor(name, age, program, languages) {  
    this.name = name;  
    this.age = age;  
    this.program = program;  
    this.languages = languages;  
  }  
  showLanguages() {  
    console.log(`${this.name} (${this.age}, ${this.program}) beherrscht  
      diese Sprachen: ${this.languages}`);  
  }  
  learnNewLanguage(language) {  
    this.languages[this.languages.length] = language;  
  }  
}
```



Klassen: Objekte

erzeugen

```
const myObject1 = new Class(arg1, arg2);  
const myObject2 = new Class(arg3, arg4);
```

Studierenden-Objekte erzeugen

```
const student1 = new Student("John Doe", 22, "MTD", ["Java", "C#"]);  
const student2 = new Student("Jane Doe", 25, "SE", ["Pascal", "C", "Java"]);  
  
student1.showLanguages(); // Java,C#  
student1.learnNewLanguage("JavaScript");  
student1.showLanguages(); // Java,C#,JavaScript  
student2.showLanguages(); // Pascal,C,Java
```

Vordefinierte Objekte



Object, Number, String, Date, Math, Array

Und viele weitere.

Object Objekt



Alle Objekte stammen von Object ab. Könnte so angelegt werden (ist aber ungebräuchlich)

```
const data = new Object();
```

Wichtiger sind statische Methoden

```
const person = { name: "John", age: 22 };  
console.log(Object.keys(person)); // ["name", "age"]  
console.log(Object.values(person)); // ["John", 22]  
console.log(Object.entries(person)); // [["name", "John"], ["age", 22]]
```



Number Objekt

Könnte mit new angelegt werden

```
const number1 = new Number(12.3456);  
const number1rounded = number1.toFixed(2);  
console.log(number1rounded); // 12.35
```

Besser aber einfach so

```
const number2 = 12.3456;  
const number2rounded = number2.toFixed(2);  
console.log(number2rounded); // 12.35
```

Nützliche Eigenschaften

```
console.log(Number.MAX_VALUE); // größte darstellbare Zahl  
console.log(Number.MIN_VALUE); // kleinste darstellbare Zahl
```

String Objekt



```
const text = "Das ist ein langer Text";
```

Zeichen an Position ausgeben

```
console.log(text.charAt(4)); // i - entspricht text[4]
```

Prüfen, ob "langer" enthalten ist (gibt true/false)

```
console.log(text.includes("langer")); // true
```

Substring: Start (inkl.) bis Ende (exkl.)

```
console.log(text.substring(4, 7)); // ist
```

Slice: Die letzten 4 Zeichen ausschneiden (Negativer Index)

```
console.log(text.slice(-4)); // Text
```

String bei den Leerzeichen teilen

```
console.log(text.split(" ")); // ["Das", "ist", "ein", "langer", "Text"]
```

Länge des Strings

```
console.log(text.length); // 23
```



Date Objekt

```
const d = new Date();
```

Komplettes Datum ausgeben (entspricht d.toString())

```
console.log(d);
```

```
// z.B. Fri Jan 09 2026 13:29:12 GMT+0100 (Central European Standard Time)
```

Jahr, Wochentag, Stunde und Minute einzeln ausgeben

```
console.log(d.getFullYear()); // z.B. 2026
```

```
console.log(d.getDay()); // z.B. 5 für Freitag
```

```
console.log(d.getHours()); //z.B. 13
```

```
console.log(d.getMinutes()); // z.B. 29
```

Formatiert für Österreich ausgeben

```
const options = { weekday: "long", year: "numeric", month: "long", day: "numeric" };
```

```
console.log(d.toLocaleDateString("de-AT", options)); // z.B. Freitag, 9. Januar 2026
```

Unix-Timestamp

```
console.log(Date.now()); // z.B. 1767961752219
```



Math Objekt

Konstanten

```
console.log(Math.PI); // 3.141592653589793
```

Runden

```
console.log(Math.round(4.6)); // 5
```

```
console.log(Math.floor(4.9)); // 4
```

Zufallszahl zwischen 1 und 10 (Ganzzahl)

```
const randomInt = Math.floor(Math.random() * 10) + 1;
```

```
console.log(randomInt); // z.B. 2
```

Maximum ermitteln (Spread-Operator '...' entpackt das Array)

```
const values = [10, 42, 5, 100];
```

```
console.log(Math.max(...values)); // 100
```

Array Objekt



Array mit 5 Einträgen

```
const grades = [1, 2, 3, 4, 5];
```

← Oldschool: `const grades = new Array(1, 2, 3, 4, 5);`

1. Element hinzufügen

```
grades.push(1);
```

← `push()` verändert das Original-Array

```
console.log(grades); // [1, 2, 3, 4, 5, 1]
```

2. Filtern (nur positive Noten ≤ 4)

```
const passed = grades.filter(n => n <= 4);
```

← Arrow-Function. `filter()` erwartet für jeden Wert `true` oder `false`

```
console.log(passed); // [1, 2, 3, 4, 1]
```

3. Transformieren (Noten verdoppeln – Mapping)

```
const doubled = passed.map(n => n * 2);
```

← Arrow-Function. `map()` erwartet für jeden Wert einen neuen Wert

```
console.log(doubled); // [2, 4, 6, 8, 2]
```

4. Prüfen ob enthalten

```
console.log(grades.includes(5)); // true
```

for...in : Objekte iterieren



```
const student = {  
  name: "John Doe",  
  age: 22,  
  program: "MTD",  
  languages: ["Java", "C#"],  
};  
  
for (const property in student) {  
  console.log("Eigenschaft: " + property);  
  console.log("Wert: " + student[property]);  
}
```

Output in der Konsole:

```
Eigenschaft: name  
Wert: John Doe  
Eigenschaft: age  
Wert: 22  
Eigenschaft: program  
Wert: MTD  
Eigenschaft: languages  
Wert: (2) ['Java', 'C#']
```

JS-Objekte und JSON



```
const rezept = {  
  "gericht": "Der klassische Tiroler Knödel",  
  "zutaten": [  
    {  
      "ingredienz": "Semmelwürfel",  
      "menge": 350,  
      "einheit": "g"  
    },  
    {  
      "ingredienz": "Milch",  
      "menge": 200,  
      "einheit": "ml"  
    }  
  ],  
  "zubereitung": [  
    "Semmelwürfel mit Milch und Eiern vermengen und 20 bis  
    30 Minuten ziehen lassen.",  
    "Kleinwürfelig geschnittenen Speck und fein gewiegte  
    Bergsteigerwurst in zerlassener Butter gemeinsam mit  
    Zwiebeln, Petersilie und Schnittlauch anbraten und  
    mit Salz und Muskatnuss abschmecken. Mehl darüber  
    streuen und aus allen Zutaten eine feste, aber nicht  
    zu feuchte Masse kneten."  
  ]  
};
```

Zugriff in JavaScript:

Zugriff auf einfache Eigenschaft

```
console.log(rezept.gericht);
```

Zugriff auf Array an Index 0, davon Eigenschaft 'ingredienz'

```
console.log(rezept.zutaten[0].ingredienz);
```

Zugriff auf Array an Index 1, davon Eigenschaft 'menge'

```
console.log(rezept.zutaten[1].menge);
```

Zugriff auf zweiten Zubereitungsschritt

```
console.log(rezept.zubereitung[1]);
```

JavaScript im Browser

Einbindung und wichtige Objekte

Einbinden in HTML: Inline



```
<!DOCTYPE html>
<html lang="de">
  <head>
    <meta charset="utf-8">
    <title>JavaScript &ndash; Inline-Einbindung</title>
  </head>
  <body>
    <div id="output"></div>
    <script>
      document.getElementById("output").innerHTML = "<p>Hallo Welt!</p>";
    </script>
  </body>
</html>
```



Einbinden in HTML:

Extern
<!DOCTYPE html>

<html lang="de">

<head>

<meta charset="utf-8">

<title>JavaScript – Externe Einbindung</title>

<script src="extern.js" defer></script>

</head>

<body>

<div id="output"></div>

</body>

</html>

extern.js:

document.getElementById("output").innerHTML = "<p>Hallo Welt!</p>";

Wenn im <head>, immer mit defer!

Alternativ hier ohne defer

<script src="extern.js"></script>

Einbinden: Event-Handler



```
<!DOCTYPE html>
<html lang="de">
  <head>
    <meta charset="utf-8">
    <title>JavaScript &ndash; Einbindung per Event-Handler</title>
  </head>
  <body>
    <button type="button" onclick="alert('Hallo Welt!')">Klick mich!</button>
  </body>
</html>
```

Beliebiger JavaScript-Code (eigene Funktionen etc.)

window Objekt

Das **globale** Objekt im Browser.

Zugriff auf das Browserfenster und seine
Eigenschaften.





window im Detail

Eigenschaften

`window.document; (Obj)`

`window.history; (Obj)`

`window.location; (Obj)`

`window.screen; (Obj)`

`window.innerWidth;`

`window.innerHeight;`

Methoden

`window.open()`

`window.close()`

`window.alert(text)`

`window.scrollTo(x, y)`

`window.focus()`

Neues Fenster öffnen



```
<html lang="de">
  <head>
    <meta charset="utf-8">
    <title>window-Objekt &ndash; Beispiel</title>
    <script src="window-demo.js" defer></script>
  </head>
  <body>
    <button type="button" onclick="openWindow()">Öffne Fenster</button>
    <button type="button" onclick="closeWindow()">SchlieÙe Fenster</button>
    <button type="button" onclick="checkOpen()">Überprüfe Fenster</button>
  </body>
</html>
```

Neues Fenster öffnen



```
let myWindow = null;

function openWindow() {
    myWindow = window.open("window.html", "NewWindow", "width=300, height=200");
    if (myWindow) {
        myWindow.focus();
    }
}

function closeWindow() {
    if (myWindow && !myWindow.closed) {
        myWindow.close();
    }
}

function checkOpen() {
    alert(myWindow && !myWindow.closed ? "Fenster offen!" : "Fenster geschlossen!");
}
```

document Objekt



Die **aktuell geöffnete Seite** im Browser.

Zugriff auf die Eigenschaften und auch alle Elemente (über das DOM).



document

Details

Eigenschaften

```
document.title;  
document.URL;  
document.referrer;  
document.lastModified;
```

Methoden

```
document.getElementById()  
document.querySelector()
```

Legacy (nicht mehr verwenden)

```
document.open()  
document.write()  
document.close()
```

Dokument auslesen



```
<!DOCTYPE html>
<html lang="de">
  <head>
    <meta charset="utf-8">
    <title>Dokument-Eigenschaften auslesen</title>
    <script src="document-demo.js" defer></script>
  </head>
  <body>
    <h1>Dokument-Manipulation</h1>
    <button type="button" onclick="showInfo()">
      Infos anzeigen & Titel ändern</button>
  </body>
</html>
```

Dokument auslesen



```
function showInfo() {  
    // Zugriff auf Eigenschaften  
    console.log("URL: " + document.URL);  
    console.log("Zuletzt geändert: " + document.lastModified);  
  
    // Ändern einer Eigenschaft  
    document.title = "Neuer Titel gesetzt!";  
    alert("Der Titel des Tabs wurde geändert.");  
}
```



Nächstes Mal in WEF1VO

JavaScript DOM: Auf alle Inhalte einer Seite zugreifen

Credits

- No credits to show today.

